

Instructions for Submission

1. File Naming and Submission:
 - a. Create a **ZIP file** containing all your solution files.
 - b. Name the ZIP file as lab09_<ERP>.zip, where <ERP> is your ERP.
 - c. Ensure that the ZIP file contains:
 - i. C++ source files
 - ii. No unnecessary files (such as executables, temporary files, or editor configs)
 - d. Do not include TEXT(.txt) files.
2. Code Neatness:
 - a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
 - b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
 - a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
 - b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

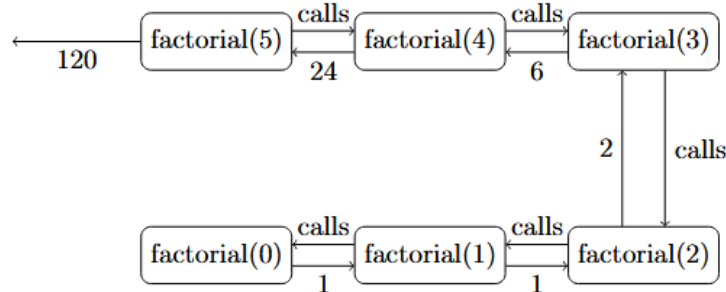
Recursive functions

A function in C++ is called recursive if it calls itself. Recursion is a powerful technique in programming. It provides a way to break complex problems into simpler ones. The recursive function calls itself with a smaller input, and the function keeps calling itself until it reaches the base case. The base case is the condition where the function stops calling itself and returns a value.

As an example, consider the following recursive function that calculates the factorial of a number:

```
1 int factorial(int n) {
2     if (n == 0) return 1;      // base case
3     return n * factorial(n - 1); // reduction step
4 }
```

Following call graph shows the sequence of function calls for **factorial(5)**, the return values are shown on the return edges:



The first call to **factorial(5)** results in a sequence of function calls until the base case is reached. The base case is when $n == 0$, and the function returns 1. The return values are then propagated back to the original call, and the final result is calculated.

char and string functions

To compute a **substring** of a given **string str**, the **substr()** function can be used. The first argument of the function is the starting index of the substring, and the second argument is the length of the substring.

Here are few examples (using `str = "Hello World"`):

- `str.substr(1)` returns a substring of `str` starting from index 1 till the end of the string. "ello World"
- `str.substr(1, 3)` returns a substring of `str` starting from index 1 and of length 3. "ell"
- `str.substr(1, str.length()-2)` returns a substring of `str` starting from index 1 and of length `str.length()-2`. "ello Worl"

Lab Tasks

Task 1: Array Summation

Write a function that computes the sum of all elements in a 1-dimensional array and returns the result.
Define the function with the following signature:

```
int sumArray(int arr[], int n);
```

The function should:

- Take an integer array `arr[]` and its size `n` as input.
- Return the sum of all elements in the array.

Call this function from `main()` with a sample array to test its behavior.

Task 2: Email Verification for IBA Students

IBA student email addresses follow a strict naming format that combines a name and an ERP number, followed by the official domain.

Each valid email must satisfy the following conditions:

Component	Rule
Name Part	Contains only lowercase letters and dots (e.g., s.raza, samreen.kazi)
ERP Part	Exactly 5 digits (e.g., 32448)
Domain	Must end with @khi.iba.edu.pk

Examples:

- s.raza.32448@khi.iba.edu.pk → Valid
- samreen.kazi.12345@khi.iba.edu.pk → Valid
- s.raza32448@khi.iba.edu.pk → Invalid (missing dot before ERP)
- s.raza.32448@iba.edu.pk → Invalid (wrong domain)
- s.Taha.29208@iba.edu.pk → Invalid (wrong domain)+Capital Letter Not allowed

Write a C++ program that verifies whether email addresses follow the correct IBA format.

1. `bool isValidIBAEmail(string email);`

This function should:

- Extract the **local part** (portion before @) using `substr()`.
- Use `rfind('.')` to locate the ERP separator.
- Use `<cctype>` functions (`isdigit()`, `islower()`) to validate each section.
- Return true if all rules are met; otherwise, return false.

2. **In `main()`:**

- Declare a string array containing at least **five sample email addresses**.
- For each email, print whether it is **Valid** or **Invalid** based on your function result.

Category	Functions / Methods
String Handling	substr(), rfind(), size()
Character Checks	isdigit(), islower() from <cctype>

Expected Output Example

Checking IBA email validity:

s.raza.32448@khi.iba.edu.pk --> Valid

samreen.kazi.12345@khi.iba.edu.pk --> Valid

Firstname.lastname@khi.iba.edu.pk --> Invalid

s.raza32448@khi.iba.edu.pk --> Invalid

s.Taha.29208@iba.edu.pk -->Invalid

//Add Comments in code to answer following questions.

1. Which string method did you find most useful and why?
2. How could you modify this program to handle faculty emails (@iba.edu.pk)?

Example Code:

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;
int main() {
    string str = "Hello World";
    cout << "Example of substr():\n";
    cout << str.substr(1) << endl;           // "ello World"
    cout << str.substr(1, 3) << endl;         // "ell"
    cout << str.substr(1, str.length()-2) << endl; // "ello Worl"
    cout << "\nExamples of <cctype> checks:\n";
    char ch = 'A';
    cout << isalpha(ch) << " " << isdigit(ch) << " " << islower(ch) << " " << isupper(ch) << endl;
}
```

Task 3: Recursive String Reversal

Write a function that reverses a string recursively **in place**.

Define the function with the following signature:

```
void reverse_string(std::string& str, int start = 0, int end = -1);
```

The function should:

- Take a reference to a `std::string` and indices `start` and `end` as input.
- Reverse the string in place recursively.
- Not return any value.

Call this function from `main()` with a sample string to verify that the reversal works correctly.

Task 4: Maximum element

Write a recursive method to find maximum element in a given array. Implement the following recursive function.

```
int findMax(const vector<int>& a) {  
    return findMax(a, a.size());  
}  
// This helper function finds the maximum of first n elements in the array  
int findMax(const vector<int>& a, int n) {  
    if(n==____)  
        return ____; // base case  
    int m = ____; // recursively find the maximum of n-1 elements  
    return ____; // return maximum of m and the last element  
}
```

Task 5: Palindrome

Write a recursive function **isPalindrome()** that returns true if and only if the input word is palindrome (a palindrome is a word, number, phrase, or other sequence of characters which reads the same backward as forward, such as "Madam" or "Racecar" or the number "10201".)

Hint: Ensure that the first and the last letters are the same, and then recursively check the rest of the word. Use the **substr()** function to get the substring of a given string (examples at the beginning)